

OPERATING SYSTEMS AND SYSTEM PROGRAMMING

SKILL – WEEK 3

Name: S Svara Haasini

Roll No: 2520090170

Task 1: Single Quotes

Question:

To Apply Single Quotes, Preserve Literal Content, Ignore Variable Expansion, Store Quoted Strings, Validate Parsing Results, Test Edge Cases

Source Code:

```

haasini@SSH:~/OSSP_upload/Skill_Week3$ cat > quote_parser.c <<'EOF'
#include <stdio.h>
#include <string.h>
#include <stdlib.h>

#define MAX_INPUT 256
#define MAX_TOKENS 20
#define TOKEN_SIZE 128

int parse_quotes(const char *input, char tokens[][TOKEN_SIZE])
{
    int count = 0;
    int i = 0;

    while (input[i] != '\0')
    {
        while (input[i] == ' ' || input[i] == '\t')
            i++;

        if (input[i] == '\0')
            break;

        if (count >= MAX_TOKENS)
            break;

        int j = 0;

        if (input[i] == '\')
        {
            i++;

            while (input[i] != '\0' && input[i] != '\')
            {
                if (j < TOKEN_SIZE - 1)
                    tokens[count][j++] = input[i];

                i++;
            }

            if (input[i] == '\')
                i++;

            EOF return 0;quote parser terminated successfully.\n");]);

```

Single-Quote Parsing Source Code

Output:

```

haasini@SSH:~/OSSP_upload/Skill_Week3$ gcc -Wall -Wextra quote_parser.c -o quote_parser
haasini@SSH:~/OSSP_upload/Skill_Week3$ ./quote_parser
=== Quote Parsing Demo ===
Type 'exit' to stop.

quote_parser> echo 'hello world'
Token count: 2
Token[0] = "echo"
Token[1] = "hello world"

quote_parser> echo '$HOME'
Token count: 2
Token[0] = "echo"
Token[1] = "$HOME"

quote_parser> echo "hello world"
Token count: 2
Token[0] = "echo"
Token[1] = "hello world"

quote_parser> echo "$HOME"
Token count: 2
Token[0] = "echo"
Token[1] = "$HOME"

quote_parser> printf "OSSP Skill Week 3"
Token count: 2
Token[0] = "printf"
Token[1] = "OSSP Skill Week 3"

quote_parser> exit
Quote parser terminated successfully.

```

Single-Quote Parsing Output

Observations:

- The program `quote_parser.c` was compiled with `gcc -Wall -Wextra` and run interactively as `./quote_parser`.
- For the input **echo 'hello world'**, the parser produced `Token[0] = "echo"` and `Token[1] = "hello world"`, showing that the space inside the single-quoted text was preserved and the quoted content was stored as a single token.
- For the input **echo '\$HOME'**, the parser produced `Token[1] = "$HOME"`. The characters `$HOME` were preserved literally by the parser; no shell variable expansion was performed.
- This confirms the single-quote handling logic in `parse_quotes()` correctly detects a leading single quote, copies characters until the matching closing quote into one token, and leaves the enclosed text unmodified.

Task 2: Double Quotes

Question:

To Apply Double Quotes, Preserve Spaces, Allow Variable Expansion, Parse Nested Tokens, Validate Outputs, Test Quoted Commands

Source Code:

```

haasini@SSH:~/OSSP_upload/Skill_Week3$ cat > quote_parser.c <<'EOF'
#include <stdio.h>
#include <string.h>
#include <stdlib.h>

#define MAX_INPUT 256
#define MAX_TOKENS 20
#define TOKEN_SIZE 128

int parse_quotes(const char *input, char tokens[][TOKEN_SIZE])
{
    int count = 0;
    int i = 0;

    while (input[i] != '\0')
    {
        while (input[i] == ' ' || input[i] == '\t')
            i++;

        if (input[i] == '\0')
            break;

        if (count >= MAX_TOKENS)
            break;

        int j = 0;

        if (input[i] == '\\')
        {
            i++;

            while (input[i] != '\0' && input[i] != '\\')
            {
                if (j < TOKEN_SIZE - 1)
                    tokens[count][j++] = input[i];

                i++;
            }

            if (input[i] == '\\')
                i++;
        }

        EOF return 0;quote parser terminated successfully.\n");]);

```

Double-Quote Parsing Source Code (quote_parser.c)

Output:

```

haasini@SSH:~/OSSP_upload/Skill_Week3$ gcc -Wall -Wextra quote_parser.c -o quote_parser
haasini@SSH:~/OSSP_upload/Skill_Week3$ ./quote_parser
=== Quote Parsing Demo ===
Type 'exit' to stop.

quote_parser> echo 'hello world'
Token count: 2
Token[0] = "echo"
Token[1] = "hello world"

quote_parser> echo '$HOME'
Token count: 2
Token[0] = "echo"
Token[1] = "$HOME"

quote_parser> echo "hello world"
Token count: 2
Token[0] = "echo"
Token[1] = "hello world"

quote_parser> echo "$HOME"
Token count: 2
Token[0] = "echo"
Token[1] = "$HOME"

quote_parser> printf "OSSP Skill Week 3"
Token count: 2
Token[0] = "printf"
Token[1] = "OSSP Skill Week 3"

quote_parser> exit
Quote parser terminated successfully.

```

Double-Quote Parsing Output

Observations:

- For the input **echo "hello world"**, the parser produced `Token[0] = "echo"` and `Token[1] = "hello world"`, showing that the space inside the double-quoted text was preserved and the quoted text remained a single token, the same as with single quotes.
- For the input **echo "\$HOME"**, the parser produced `Token[1] = "$HOME"`. This demonstration parser does not perform actual environment-variable substitution; it stores the double-quoted content as a literal quoted token. Real shell-style variable expansion would be a subsequent processing step not implemented here.
- For the input **printf "OSSP Skill Week 3"**, the multi-word double-quoted argument was stored as one token, `Token[1] = "OSSP Skill Week 3"`, confirming correct parsing of quoted commands containing several space-separated words.

Task 3: Escape Sequences and Special Characters

Question:

To Create Process Escape Sequences, Handle Escaped Spaces, Escape Special Symbols, Preserve Characters, Validate Parser Output, Test Complex Inputs

Source Code:

```

haasini@SSH:~/OSSP_upload/Skill_Week3$ cat escape_exec.c
#include <stdio.h>
#include <stdlib.h>
#include <string.h>
#include <unistd.h>
#include <sys/types.h>
#include <sys/wait.h>

#define MAX_INPUT 256
#define MAX_ARGS 20
#define ARG_SIZE 128

int parse_escaped(char *input, char args[][ARG_SIZE])
{
    int count = 0;
    int i = 0;

    while (input[i] != '\0')
    {
        while (input[i] == ' ' || input[i] == '\t')
            i++;

        if (input[i] == '\0')
            break;

        int j = 0;

        while (input[i] != '\0')
        {
            if (input[i] == '\\' && input[i + 1] != '\0')
            {
                i++;

                if (j < ARG_SIZE - 1)
                    args[count][j++] = input[i];

                i++;
            }
            else if (input[i] == ' ' || input[i] == '\t')
            {

```

Escape-Sequence Parsing Source Code (escape_exec.c, part 1)

```
haasini@SSH: ~/OSSP_upload × + v

    else if (input[i] == ' ' || input[i] == '\t')
    {
        break;
    }
    else
    {
        if (j < ARG_SIZE - 1)
            args[count][j++] = input[i];

        i++;
    }
}

args[count][j] = '\0';
count++;

if (count >= MAX_ARGS)
    break;
}

return count;
}

int execute_command(char args[][ARG_SIZE], int count)
{
    char *argv[MAX_ARGS + 1];

    for (int i = 0; i < count; i++)
        argv[i] = args[i];

    argv[count] = NULL;

    pid_t pid = fork();

    if (pid < 0)
    {
        perror("fork");
        return -1;
    }

    if (pid == 0)
    {
        printf("Child PID: %d\n", getpid());
        printf("Parent PID: %d\n", getppid());

        execvp(argv[0], argv);

        perror("execvp");
        _exit(127);
    }

    printf("Parent PID: %d\n", getpid());
    printf("Created Child PID: %d\n", pid);

    int status;

    waitpid(pid, &status, 0);

    if (WIFEXITED(status))
    {
        printf("Child exit status: %d\n",
               WEXITSTATUS(status));
    }

    return 0;
}
```

Escape-Sequence Parsing Source Code (escape_exec.c, part 2)

Output:

```

haasini@SSH:~/OSSP_upload/Skill_Week3$ gcc -Wall -Wex
haasini@SSH:~/OSSP_upload/Skill_Week3$ ./escape_exec
=== Escape and Process Execution Demo ===
Type 'exit' to stop.

exec_demo> echo hello\ world
Parsed arguments:
argv[0] = "echo"
argv[1] = "hello world"
Parent PID: 1973
Created Child PID: 1974
Child PID: 1974
Parent PID: 1973
hello world
Child exit status: 0

exec_demo> echo OSSP\ Skill\ Week\ 3
Parsed arguments:
argv[0] = "echo"
argv[1] = "OSSP Skill Week 3"
Parent PID: 1973
Created Child PID: 1975
Child PID: 1975
Parent PID: 1973
OSSP Skill Week 3
Child exit status: 0

exec_demo> printf %s\n hello
Parsed arguments:
argv[0] = "printf"
argv[1] = "%s\n"
argv[2] = "hello"
Parent PID: 1973
Created Child PID: 1978
Child PID: 1978
Parent PID: 1973
hellonChild exit status: 0

exec_demo> nosuchcommand
Parsed arguments:
argv[0] = "nosuchcommand"
Parent PID: 1973
Child PID: 1979
Created Child PID: 1979
Parent PID: 1973
execvp: No such file or directory
Child exit status: 127

exec_demo> exit
Execution demo terminated successfully.
haasini@SSH:~/OSSP_upload/Skill_Week3$ cat escape_exe

```

Escape-Sequence Parsing Output

Observations:

- The function `parse_escaped()` scans the input character by character. When it encounters a backslash followed by a non-null character, it skips the backslash and copies the following character directly into the current argument instead of treating it as a token separator.

- For the input **echo hello\ world**, the output shows `argv[0] = "echo"` and `argv[1] = "hello world"` — the escaped space was preserved inside a single argument rather than splitting it into two.
- For the input **echo OSSP\ Skill\ Week\ 3**, the output shows `argv[1] = "OSSP Skill Week 3"`, confirming that multiple escaped spaces in one input are all correctly merged into a single argument.
- For the input **printf %s\n hello**, the output shows `argv[1] = "%s\n"` and `argv[2] = "hello"`. The backslash before 'n' was consumed by the escape handling and the following character 'n' was preserved literally as part of the argument, rather than being interpreted as a newline control character.
- These runs show complex inputs being split correctly into `argv` entries by the parser, with the reconstructed arguments printed back as "Parsed arguments" before execution.

Task 4: Child Process and Command Execution

Question:

To Create Child Processes, Execute Programs, Handle Execution Errors, Pass Arguments, Manage Parent Process, Test Command Launching

Source Code:

```

haasini@SSH:~/OSSP_upload/Skill_Week3$ cat escape_exec.c
#include <stdio.h>
#include <stdlib.h>
#include <string.h>
#include <unistd.h>
#include <sys/types.h>
#include <sys/wait.h>

#define MAX_INPUT 256
#define MAX_ARGS 20
#define ARG_SIZE 128

int parse_escaped(char *input, char args[][ARG_SIZE])
{
    int count = 0;
    int i = 0;

    while (input[i] != '\0')
    {
        while (input[i] == ' ' || input[i] == '\t')
            i++;

        if (input[i] == '\0')
            break;

        int j = 0;

        while (input[i] != '\0')
        {
            if (input[i] == '\\' && input[i + 1] != '\0')
            {
                i++;

                if (j < ARG_SIZE - 1)
                    args[count][j++] = input[i];

                i++;
            }
            else if (input[i] == ' ' || input[i] == '\t')
            {

```

Command Execution Source Code (escape_exec.c, part 1)

```
haasini@SSH: ~/OSSP_upload × + v

    else if (input[i] == ' ' || input[i] == '\t')
    {
        break;
    }
    else
    {
        if (j < ARG_SIZE - 1)
            args[count][j++] = input[i];

        i++;
    }
}

args[count][j] = '\0';
count++;

if (count >= MAX_ARGS)
    break;
}

return count;
}

int execute_command(char args[][ARG_SIZE], int count)
{
    char *argv[MAX_ARGS + 1];

    for (int i = 0; i < count; i++)
        argv[i] = args[i];

    argv[count] = NULL;

    pid_t pid = fork();

    if (pid < 0)
    {
        perror("fork");
        return -1;
    }

    if (pid == 0)
    {
        printf("Child PID: %d\n", getpid());
        printf("Parent PID: %d\n", getppid());

        execvp(argv[0], argv);

        perror("execvp");
        _exit(127);
    }

    printf("Parent PID: %d\n", getpid());
    printf("Created Child PID: %d\n", pid);

    int status;

    waitpid(pid, &status, 0);

    if (WIFEXITED(status))
    {
        printf("Child exit status: %d\n",
               WEXITSTATUS(status));
    }

    return 0;
}
```

Command Execution Source Code (escape_exec.c, part 2 – execute_command())

Output:

```

haasini@SSH:~/OSSP_upload/Skill_Week3$ gcc -Wall -Wex
haasini@SSH:~/OSSP_upload/Skill_Week3$ ./escape_exec
=== Escape and Process Execution Demo ===
Type 'exit' to stop.

exec_demo> echo hello\ world
Parsed arguments:
argv[0] = "echo"
argv[1] = "hello world"
Parent PID: 1973
Created Child PID: 1974
Child PID: 1974
Parent PID: 1973
hello world
Child exit status: 0

exec_demo> echo OSSP\ Skill\ Week\ 3
Parsed arguments:
argv[0] = "echo"
argv[1] = "OSSP Skill Week 3"
Parent PID: 1973
Created Child PID: 1975
Child PID: 1975
Parent PID: 1973
OSSP Skill Week 3
Child exit status: 0

exec_demo> printf %s\n hello
Parsed arguments:
argv[0] = "printf"
argv[1] = "%s\n"
argv[2] = "hello"
Parent PID: 1973
Created Child PID: 1978
Child PID: 1978
Parent PID: 1973
hellonChild exit status: 0

exec_demo> nosuchcommand
Parsed arguments:
argv[0] = "nosuchcommand"
Parent PID: 1973
Child PID: 1979
Created Child PID: 1979
Parent PID: 1973
execvp: No such file or directory
Child exit status: 127

exec_demo> exit
Execution demo terminated successfully.
haasini@SSH:~/OSSP_upload/Skill_Week3$ cat escape_exe

```

Parent-Child Execution Output

Observations:

- In `execute_command()`, `fork()` creates a child process. In the child branch (`pid == 0`), the program prints its own PID via `getpid()` and its parent's PID via `getppid()`, then calls `execvp(argv[0], argv)` to execute the requested program.

- For **echo hello\ world**, the output shows Parent PID: 1973 and Created Child PID: 1974, with the child reporting Child PID: 1974 and Parent PID: 1973 — confirming the parent and child have distinct PIDs and the child correctly identifies its parent.
- Arguments built by the escape parser are stored in the `args[]` array, copied into an `argv[MAX_ARGS + 1]` array with a NULL terminator, and passed to `execvp()`. For **echo OSSP\ Skill\ Week\ 3**, a new child (PID 1975) is created and the program outputs "OSSP Skill Week 3" with Child exit status: 0.
- The parent process calls `waitpid(pid, &status, 0)` to wait for the child to finish, then checks `WIFEXITED(status)` and prints the exit code via `WEXITSTATUS(status)`, as seen in each "Child exit status" line.
- For the invalid command **nosuchcommand**, `execvp()` fails: the output shows "execvp: No such file or directory", produced by `perror("execvp")` in the child branch, followed by Child exit status: 127, matching the `_exit(127)` call used when `execvp()` fails.
- This confirms valid commands execute successfully with exit status 0, while an invalid command demonstrates execution-error handling with exit status 127, and the parent process obtains the child's termination status in both cases.